

ECE391

Computer System Engineering

Lecture 16

Prof. Kirill Levchenko and Dong Kai Wang

Slides Adapted from Prof. Kalbarczyk

University of Illinois at Urbana- Champaign

Spring 2024

Reminders

- **MP3 Checkpoint 1**
 - **Due by 5:59PM Tuesday, March 19 (Today)**

Lecture Topics

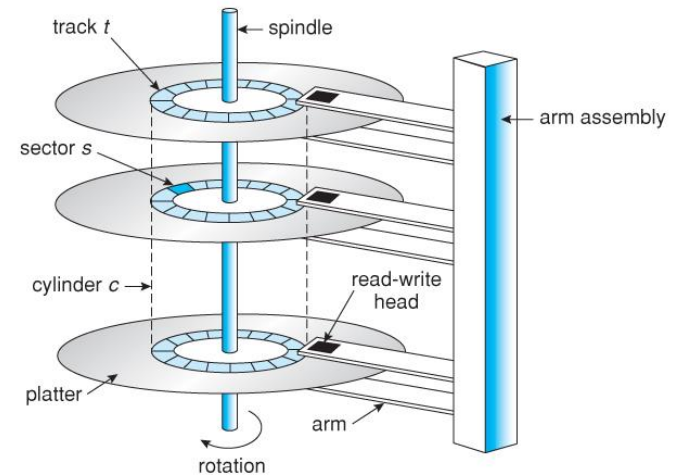
- **File Systems**

- Storage Devices
- Linux VFS
 - Data Structures: superblock, inode, dentry, file.
- File Operations
- EXT2 High-level Overview

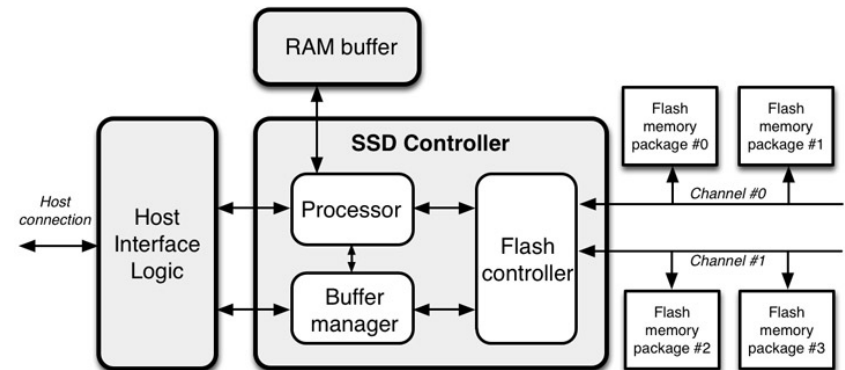
– *Recommended Readings: ULK Ch. 12 and 18.*

Storage Devices

- Q: What's the difference between *memory* and *storage*?
- Many ways to store data:
 - Magnetic
 - Voltage levels
 - Optical
 - Phase change material
 - ...



Architecture of a solid-state drive

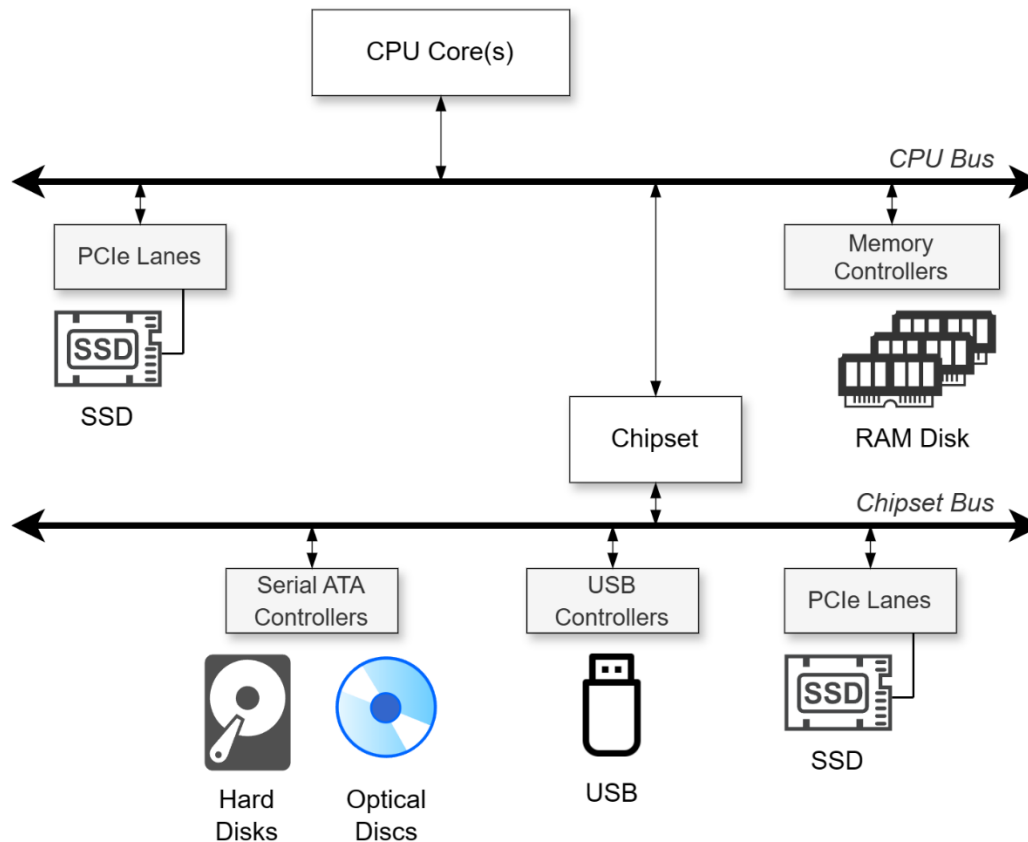


Images: (Left)
Samsung 840 Pro,
StorageReview,
(Right Top) UIC,
(Right Bottom)
Kim, et al.



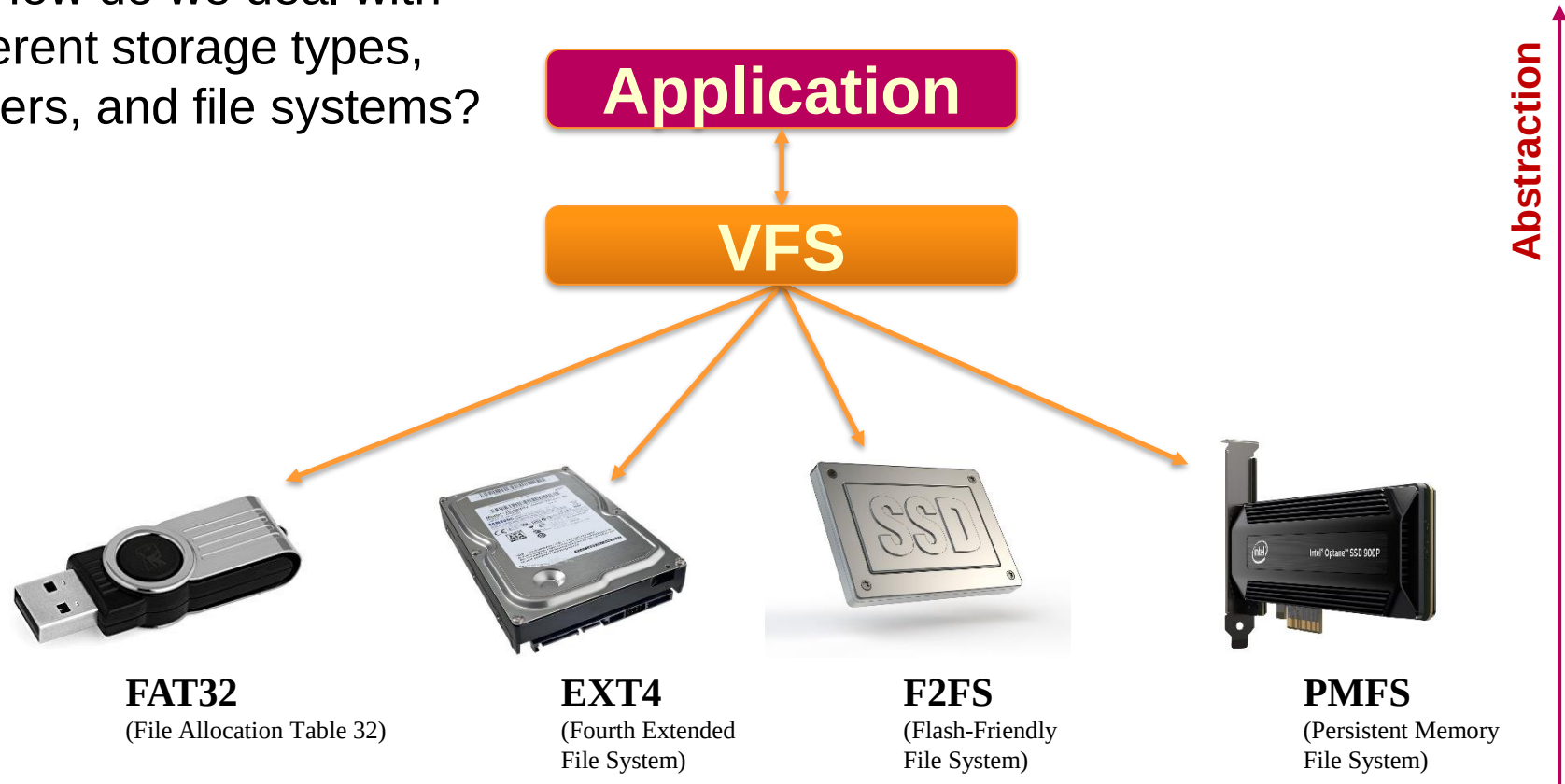
Storage Devices

- Unlike memory, storage devices are **I/O devices**.
 - A lot of variety and interfaces.
 - Not as simple as reading or writing an address, needs device drivers.



File Systems

Q: How do we deal with different storage types, drivers, and file systems?



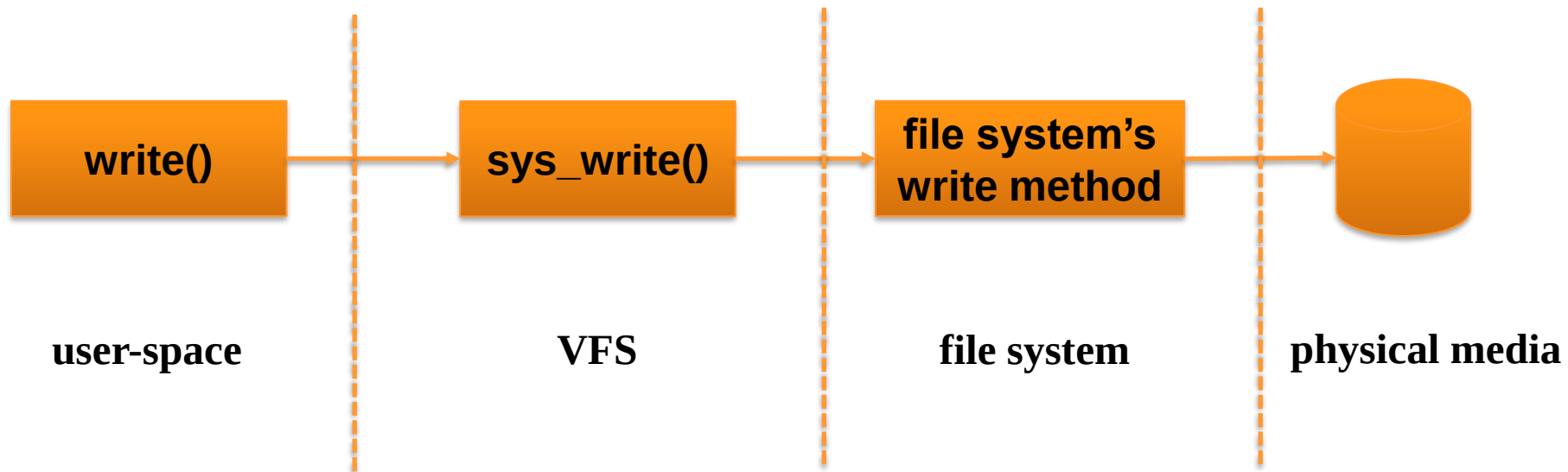
Note: File systems are hardware agnostic (*to an extent**)

File System Abstraction Layer

- Common file model abstraction.
- VFS provides common file system interfaces.
 - create, open, read, write, etc.
 - Use pointers to point to file system's corresponding functions (e.g., recall interrupt handlers).
 - Programs do not have to be aware of underlying file system when trying to open, read, write a file.
 - Abstract away the complexity of hardware.
- Each file system implements the real functionalities on top of device drivers.

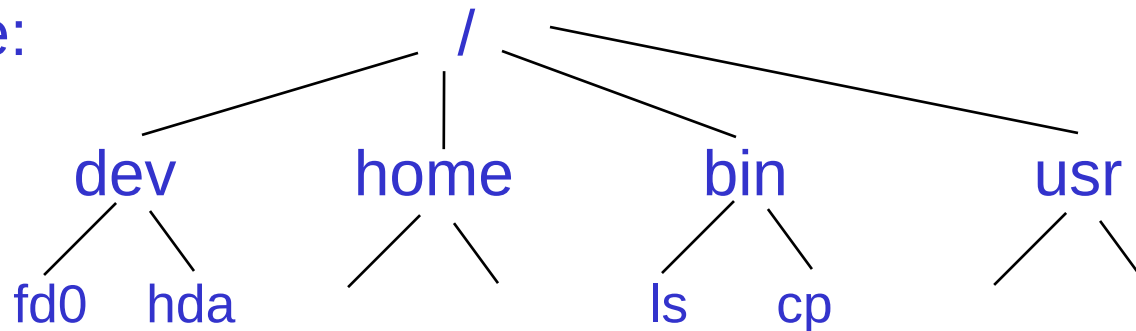
An Example

➤ `write (fd, &buf, len)`



Linux Directory Tree

- A Unix-like file system is an information container structured as a sequence of bytes.
- Files are organized in a tree structured namespace.
- Example:



- All nodes (except leaves) denote directories.
- The directory corresponding to the root is called the root directory with path [slash “/”].

File Types in Linux

- Regular files
- Directory (← Yes, directories are treated as files)
- Symbolic link
- Block-oriented device file
- Character-oriented device file
 - Covered later in lectures on device drivers.
- Pipe and named pipes
- Socket
 - Used for inter-process communications, data exchange in same system (pipes) and across network (sockets).

File Descriptor and Inode

- **File** is a sequence of bytes.
- There is clear distinction between the contents of a file and the information about a file.
 - With the exception of device files and files of special file systems, files only contain data bytes, no control information.
- **Inode (index node)** contains information needed by the file system to handle a file (size, owner, etc.).
 - Each file has its own **inode**, *it's a unique identifier*.
- **File descriptor** is an index to a data structure (table at the kernel level) containing the details of all open files.
 - Created by a process (note: per-process) when a file is opened.
 - Multiple file descriptors may be associated with a single file.
 - E.g., 0 – STDIN, 1 – STDOUT, 2 – STDERR.

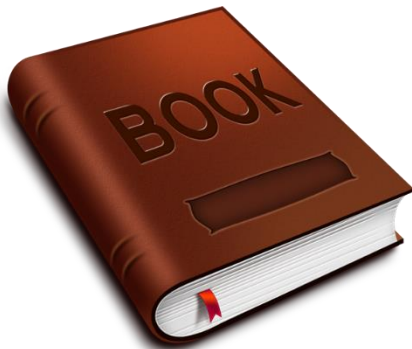
Inode Attributes (POSIX)

- **Inode** typically contains at least: (*by POSIX standard, can vary in different file systems*)
 - File type (*not file extension, see prev. two slides*).
 - File length (*bytes*).
 - Device ID (*identifier of hardware device containing file*).
 - Inode number that identifies the file within the file system.
 - UID (*User ID*) of file owner.
 - Link count of number of hard links (*not symlinks*) to inode.
 - Q: difference between symbolic and hard links?
 - Timestamps (*last access time, last modified time*).
 - etc.
 - *Note: no entry for file name!*

File Descriptor and Inode

- **Superblock** object – represents a specific mounted file system.
- **Inode** object – represents a specific file.
- **Dentry** object – represents a directory entry, a single component of a path .
- **File** object – represents an open file as associated with a process (exists only in the kernel memory). Do not confuse with file descriptor (index to table that points to file objects).
- All objects are stored in a suitable data structure.
 - Includes object attributes and a pointer to a table of object methods.

Library Analogy



Book: the **file** itself,
containing data



File descriptor a
reference no. to the book
borrowing (bad analogy)

Book ID

Book Type

Total Pages

Last Borrowed Date

Owner Name

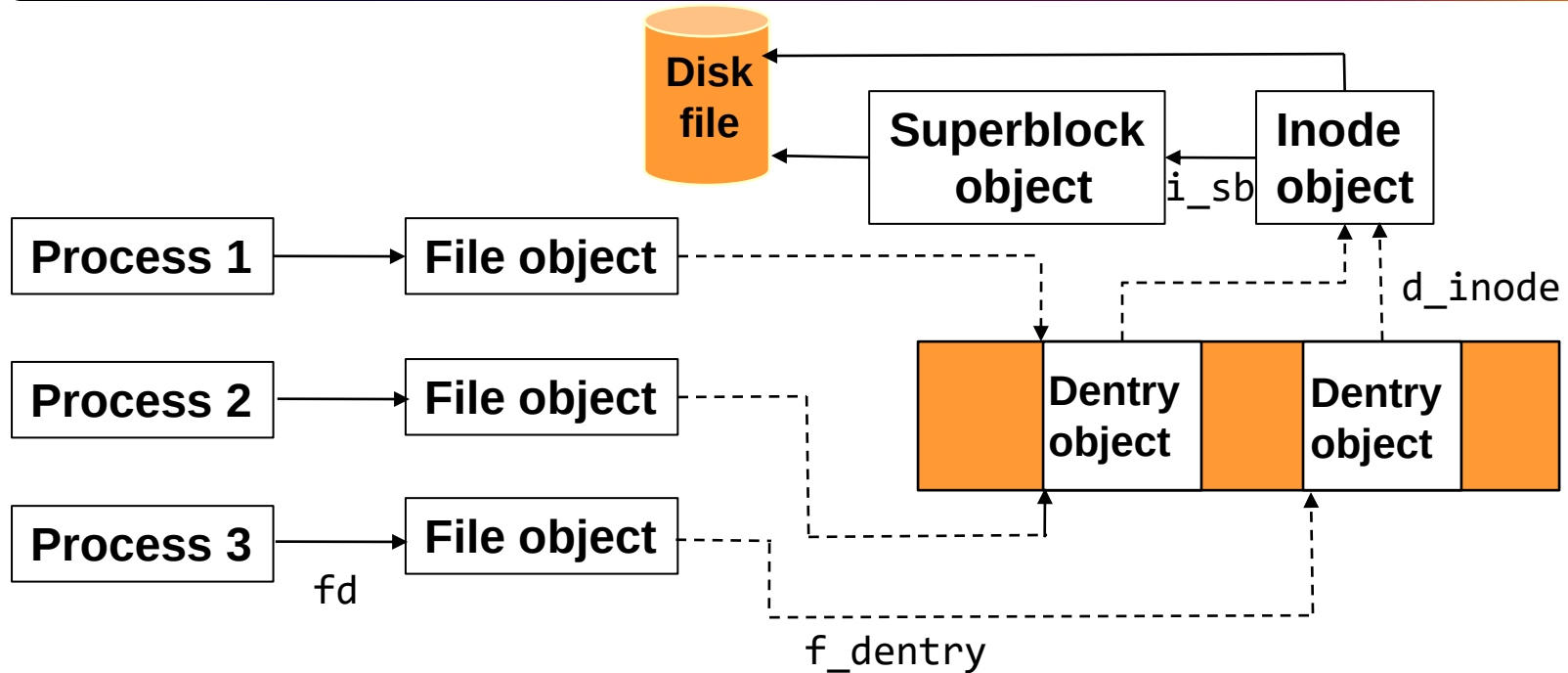
Inode is a unique
reference to book



Dentry the shelf and book
location: sign and title

Superblock contains
metadata about the
library (file system)

Interaction – Processes and File System Objects



- Processes open the same file.
- Two of them use the same link (i.e., same dentry).
- Each uses its own file object.

Some System Calls handled by VFS

System call name	Description
mount() umount() umount2()	Mount/unmount filesystems
sysfs()	Get filesystem information
statfs() fstatfs() statfs64() fstatfs64() ustat()	Get filesystem statistics
chroot() pivot_root()	Change root directory
chdir() fchdir() getcwd()	Manipulate current directory
mkdir() rmdir()	Create and destroy directories
getdents() getdents64() readdir() link() unlink() rename() lookup_dcookie()	Manipulate directory entries
readlink() symlink()	Manipulate soft links
chown() fchown() lchown() chown16() fchown16() lchown16()	Modify file owner
chmod() fchmod() utime()	Modify file attributes
stat() fstat() lstat() access() oldstat() oldfstat() oldlstat() stat64() lstat64() fstat64()	Read file status
open() close() creat() umask()	Open, close, and create files
dup() dup2() fcntl() fcntl64()	Manipulate file descriptors
select() poll()	Wait for events on a set of file descriptors
truncate() ftruncate() truncate64() ftruncate64()	Change file size
lseek() _llseek()	Change file pointer
read() write() readv() writev() sendfile() sendfile64() readahead()	Carry out file I/O operations

*Image: O'Reilly
Media.*

super_block Struct

- <https://elixir.bootlin.com/linux/2.4.10/source/include/linux/fs.h#L686>

```
struct super_block {  
    struct list_head  
    kdev_t  
    unsigned long  
    unsigned char  
    unsigned char  
    unsigned long long  
    struct file_system_type  
    struct super_operations  
    struct dquot_operations  
    unsigned long  
    unsigned long  
    struct dentry  
    struct rw_semaphore  
    struct semaphore  
    int  
    atomic_t  
    s_list;  
    s_dev;  
    s_blocksize;  
    s_blocksize_bits;  
    s_dirt;  
    s_maxbytes;  
    *s_type;  
    *s_op;  
    *dq_op;  
    s_flags;  
    s_magic;  
    *s_root;  
    s_umount;  
    s_lock;  
    s_count;  
    s_active;  
};
```

/ Keep this first */*

Doubly linked-list to all mounted file systems.

/ Max file size */*

Registered file systems with kernel.

Struct containing function pointers to file system operations, each file system provides its own functions.
(see next slide)

super_operations Struct

```
struct super_operations {  
    struct inode *(*alloc_inode)(struct super_block *sb);  
    void (*destroy_inode)(struct inode *);  
  
    void (*read_inode) (struct inode *);  
  
    void (*dirty_inode) (struct inode *);  
    void (*write_inode) (struct inode *, int);  
    void (*put_inode) (struct inode *);  
    void (*drop_inode) (struct inode *);  
    void (*delete_inode) (struct inode *);  
    void (*put_super) (struct super_block *);  
    void (*write_super) (struct super_block *);  
    int (*sync_fs)(struct super_block *sb, int wait);  
    void (*write_super_lockfs) (struct super_block *);  
    void (*unlockfs) (struct super_block *);  
    int (*statfs) (struct super_block *, struct kstatfs *);  
    int (*remount_fs) (struct super_block *, int *, char *);  
    void (*clear_inode) (struct inode *);  
    void (*umount_begin) (struct super_block *);  
  
    int (*show_options)(struct seq_file *, struct vfsmount *);  
};
```

inode Struct

```
struct inode {  
    struct list_head    i_hash;  
    struct list_head    i_list;  
    struct list_head    i_dentry; ← List of dentry objects referring to  
                                   this inode.  
  
    struct list_head    i_dirty_buffers;  
    struct list_head    i_dirty_data_buffers;  
  
    unsigned long        i_ino; ← Inode number, usage counter.  
    atomic_t            i_count;  
    kdev_t              i_dev;  
    umode_t             i_mode;  
    nlink_t             i_nlink;  
    uid_t               i_uid; ← Owner and group identifier.  
    gid_t               i_gid;  
    kdev_t              i_rdev;  
    loff_t              i_size;  
    time_t              i_atime; ← Last file access time, file write time,  
    time_t              i_mtime;  inode change time.  
    time_t              i_ctime;  
    unsigned long        i_blksize;  
    unsigned long        i_blocks;  
    unsigned long        i_version;  
    struct semaphore     i_sem;  
    struct semaphore     i_zombie;  
    struct inode_operations *i_op; ← Inode operations, file operations  
    struct file_operations *i_fop; /* former ->i_op->default_file_ops */  
    struct super_block    *i_sb; ← Superblock pointer.  
};
```

File Objects

- Recall: file object describes how a **process** interacts with a file it has opened.
 - Specific to a process (file descriptor table).
- Contains a file pointer, indicating current position in file for next operation.
 - Q: why not store in inode?

```
struct file {  
    struct list_head  f_list;  
    struct dentry     *f_dentry;  
    struct vfsmount   *f_vfsmnt;  
    struct file_operations *f_op;  
    atomic_t          f_count;  
    unsigned int       f_flags;  
    mode_t             f_mode;  
    loff_t             f_pos;  
};
```

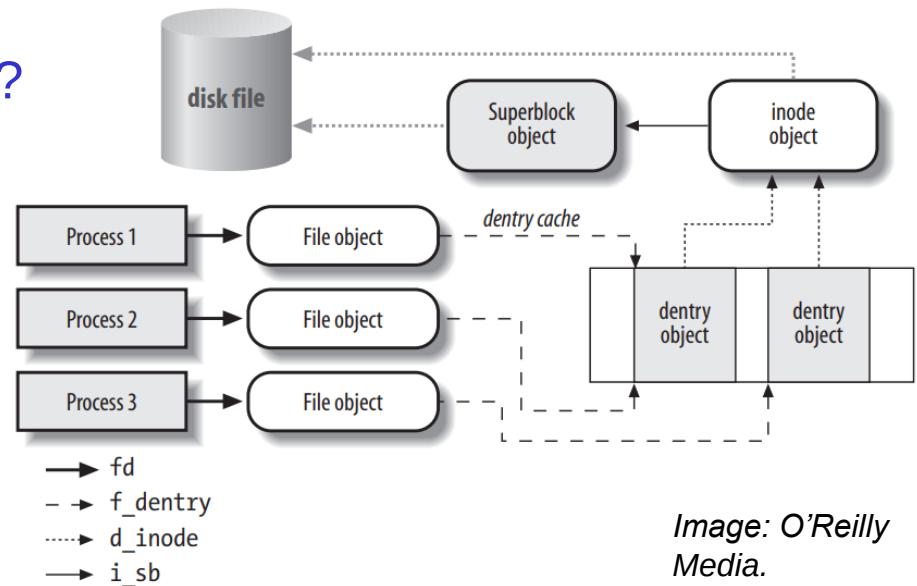


Image: O'Reilly Media.

dentry Struct

- Recall that VFS considers directories as files as well.
 - Files that contain a list of other files.
- VFS transforms directories into dentry structure.
- Kernel creates a dentry for each file:
 - e.g., /tmp/test
 - 1st dentry object: /
 - 2nd dentry object: tmp
 - 3rd dentry object: test
- dentry cache
 - Caches in-use dentries in memory.

Type	Field	Description
atomic_t	d_count	Dentry object usage counter
unsigned int	d_flags	Dentry cache flags
spinlock_t	d_lock	Spin lock protecting the dentry object
struct inode *	d_inode	Inode associated with filename
struct dentry *	d_parent	Dentry object of parent directory
struct qstr	d_name	Filename
struct list_head	d_lru	Pointers for the list of unused dentries
struct list_head	d_child	For directories, pointers for the list of directory dentries in the same parent directory
struct list_head	d_subdirs	For directories, head of the list of subdirectory dentries
struct list_head	d_alias	Pointers for the list of dentries associated with the same inode (alias)
unsigned long	d_time	Used by d_revalidate method
struct dentry_operations*	d_op	Dentry methods
struct super_block *	d_sb	Superblock object of the file
void *	d_fsdata	Filesystem-dependent data
struct rcu_head	d_rcu	The RCU descriptor used when reclaiming the dentry object (see the section "Read-Copy Update (RCU)" in Chapter 5)
struct dcookie_struct *	d_cookie	Pointer to structure used by kernel profilers
struct hlist_node	d_hash	Pointer for list in hash table entry

files_struct Struct

- Recall that each process has its own set of file objects in use.
 - Stored as an array of file object pointers (file descriptor).

```
/*  
 * Open file table structure  
 */  
struct files_struct {  
    atomic_t count;  
    rwlock_t file_lock;    /* Protects all the below members.  
    int max_fds;  
    int max_fdset;  
    int next_fd;  
    struct file ** fd;      /* current fd array */  
    fd_set *close_on_exec;  
    fd_set *open_fds;  
    fd_set close_on_exec_init;  
    fd_set open_fds_init;  
    struct file * fd_array[NR_OPEN_DEFAULT];  
};
```

Lock for modifications to fd array and other members.

File Operations

- File operations structure:
 - jump table of file operations / character driver operations.
 - generic instance for files on disk.
 - distinct instances for sockets, etc.
 - one instance per device type.

VFS Summary

- Linux **Virtual File System (VFS)**:
 - An abstraction layer for all file systems supported in Linux.
 - Uses the Linux *Common File Model*:
 - **Object**-oriented structures (structs with function pointers):
 - **Superblock**: stores file system metadata and functions.
 - **Inode**: stores metadata of a file, associated with an inode number uniquely identifying the file within the file system.
 - **File**: stores information about the interaction between an open file and a process, each process has an fd array.
 - **Dentry**: stores information about the linking of a directory entry (a filename) with the corresponding file.

VFS Summary

- *Keep in mind:*
 - File object is specific to process, different processes can access the same file, each have their own file objects in their fd array.
 - E.g., `f_pos` is current position in file being read/written by the process.
 - Inode vs dentry:
 - Inode stores metadata and attributes *intrinsic* to a file.
 - E.g. your name.
 - Dentry stores a filename, directory associated with (points to) inode.
 - Symbolic vs hard links:
 - Symlinks point to file by path.
 - `$ ln -s`
 - Hard links point to file's inode.
 - `$ ln`

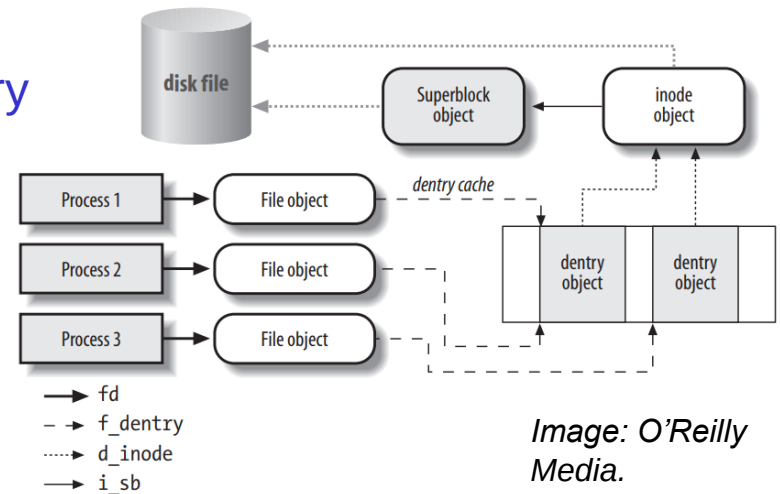


Image: O'Reilly Media.

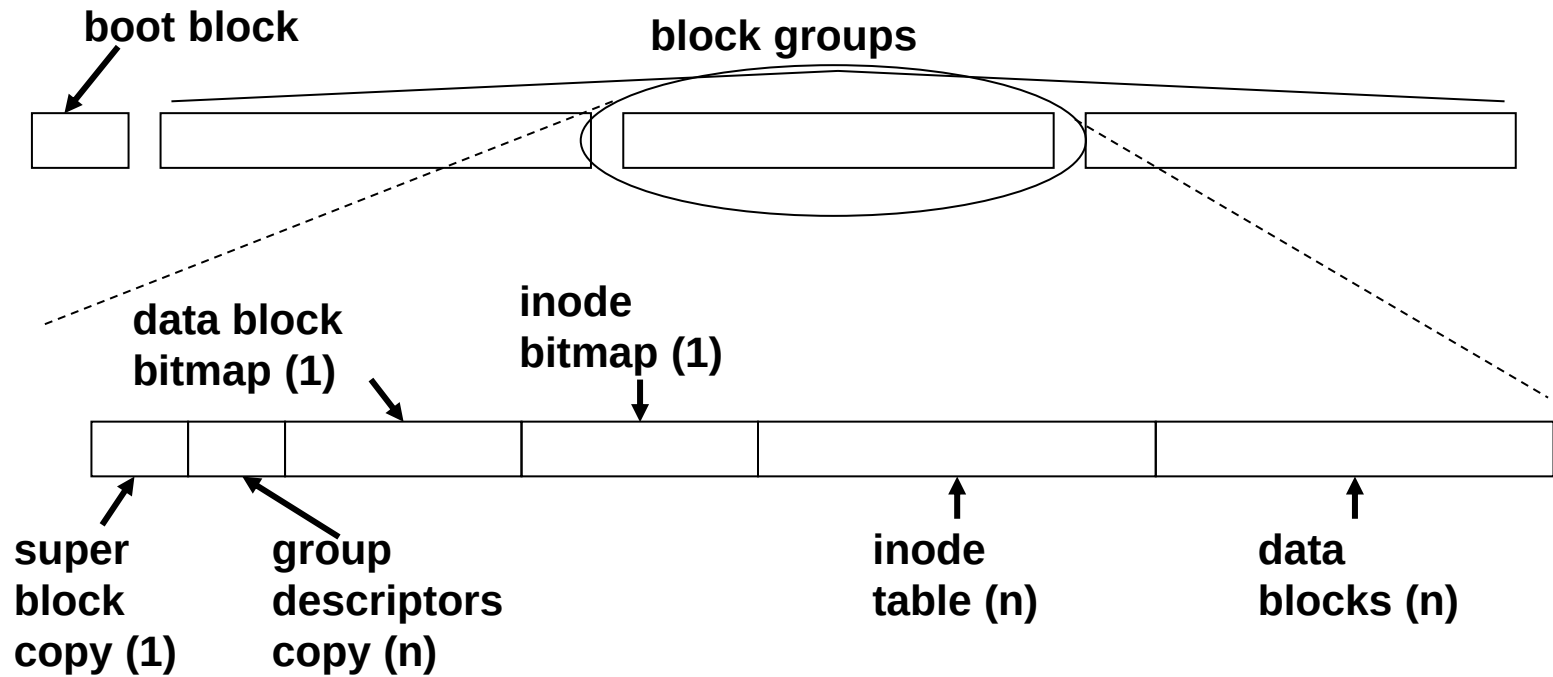
VFS and File System

- Recall VFS is an abstraction for (*all supported*) file systems in Linux.
 - Exists purely in memory, disk content is managed by the actual file system.
 - Linux kernel ***translates*** file system structures and metadata to VFS structures, recall VFS structs contain **function pointers**.
 - Not all file systems (e.g., FAT32) use inode / dentry like VFS, so kernel must translate.
 - Even in EXT2/3/4, **inodes of the file system** stored on disk are **not the same** as **inodes of VFS**.
 - Translate initially when mounting a file system (creates superblock).
 - And also on-demand when accessing files.
 - VFS dentry, inodes created on the fly (e.g., accessing /tmp/test).

File System on Disk

- Hard disks are slow:
 - Fast ones rotate at ~10,000 rpm.
 - Average of $(60\text{sec}/10000)/2 = 3$ ms to wait for disk to turn.
 - Another few milliseconds to move read head side to side.
 - 5-10 millisecond access time.
- Disks use blocks (and prefetching) to alleviate problems.
 - File systems can use several adjacent disk blocks to further improve.
 - EXT2 allows 1- 4kB blocks (chosen when file system is created).
 - Much larger access granularity than e.g., bytes for memory.

EXT2 (Second Extended File System) on Disk



EXT2 on Disk

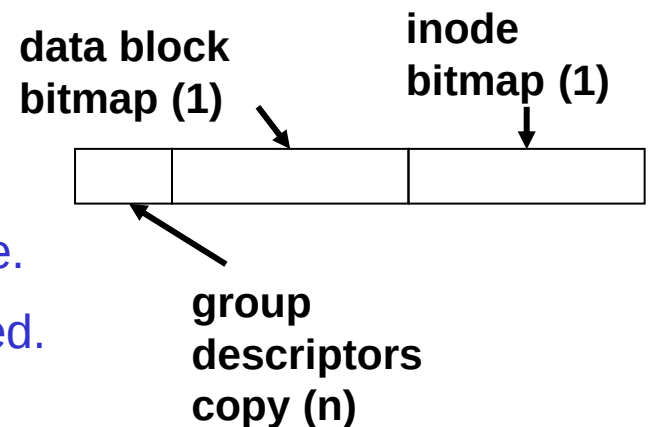
- EXT2 blocks are fixed size (1kB to 4kB) chosen at creation time.
- Groups are fixed size, also chosen at creation time.
- Two blocks replicated in each block group for reliability.
 - Super block describes file system (e.g., choice of block size).
 - Only block group 0's super block is used by kernel.
 - Group descriptors describes block groups.
- Bitmaps used to represent free blocks for index nodes (metadata) and data.

Superblock Image on Disk

- Sized at 1kB, so fits within any selected block size.
- Sizes selected for file system.
- File system check information.
 - # mounts between checks, time between checks, errors found.
 - also state of file system:
 - 0 when mounted/uncleanly dismounted.
 - 1 when cleanly dismounted.
 - 2 if errors have been found.
- Reserved blocks & authentication data.
- Volume name.
- Performance specs.

Group Descriptor and Bitmaps on Disk

- Stores information about current block group.
- Sized at 32B, so 32 of them fit within any selected block size.
- *Shortcuts* to (block numbers of) block / inode bitmaps and inode table / data blocks.
- Free block counts within current block group.
- Data block and Inode bitmaps:
 - Stored as sequence of **bits**:
 - **0**: corresponding inode or data block is free.
 - **1**: corresponding inode or data block is used.



Index Node on Disk (128B total)

`i_mode` file type & access rights (e.g., readable by group)

`i_uid` owner

`i_size` length in bytes

`i_atime` time of last access

`i_ctime` time of last creation

`i_mtime` time of last modification

`i_dtime` time of last deletion

`i_gid` group id

`i_links_count` # of hard links

`i_blocks` blocks in file (disk blocks, in 512B units)

`i_flags` e.g., immutable, append-only

`i_block[15]` data block #'s

`i_generation` generation (incremented on each modification)

File Data Blocks Stored Hierarchically

- let b denote block size.
- first 12 block pointers are direct: block #'s (first $12b$ bytes of file).
- 13th block pointer is indirect: block # of a block of block #'s ($b/4$ pointers, so the next $b^2/4$ data bytes in file).
- 14th block pointer is doubly indirect (next $b^3/16$ data bytes in file).
- 15th block pointer is triply indirect (last $b^4/64$ data bytes in file).
- small files only need direct blocks.

File Data Blocks

